Private Browser

Implementation plan for developers and AI coding agents
React Native (Expo) · Android first · Version 1.0 · 18 October 2026

Product statement. A private mobile browser that blocks ads and trackers, detects videos on the pages the user visits, and downloads them. It does what it says and nothing more.

What this document is

A build plan: scope, a numbered feature register, architecture, a security design, nine delivery phases with acceptance criteria, a data model, an extension-porting guide, a test plan and open risks. Every feature has an ID (for example DL-2). Commits, pull requests and tests should reference these IDs.

Contents

- 1. Scope and rules for AI agents
- 2. Feature register (selected features)
- 3. Added security and library requirements
- 4. Architecture and tech stack
- 5. Delivery phases 0 to 8
- 6. Data model
- 7. Porting the existing video extension
- 8. Threat model
- 9. Testing and QA
- 10. Risks and open decisions
- Appendix: agent kickoff prompt

1. Scope and rules for AI agents

In scope

- Private browsing by default: nothing persists unless the user saves it.
- Ad and tracker blocking with local lists and per-site allowlist.
- Automatic video detection and download, using the logic from the user's existing extension.
- A protected video library: encrypted database, mandatory unlock, source page saved for each video, user-controlled removal.

Out of scope (do not build)

- Accounts, sync, cloud backup, analytics, ads, crash reporting to a server.
- A browser extension store or loading Chrome/Brave extensions (not possible in a mobile WebView).
- Circumventing DRM. Protected streams are detected and marked, never downloaded.
- VPN, password manager, AI features, video editing, social or sharing features.

Rules for AI agents

- **Build only registered features.** If something looks missing, write it under 'Proposed changes' in the pull request and ask the human. Do not add it.
- **Work in phase order.** Do not start a phase until the previous phase's acceptance criteria pass. Phase 5 is blocked until the extension source code is supplied (EXT-1).
- **Security rules are non-negotiable:** the database is always encrypted; no network calls except page loads, user-initiated downloads and the blocklist update; no analytics or tracking libraries; never log URLs, page titles or video titles; never put video titles in notifications or in-progress file names.
- **Do not guess APIs.** Where this plan says 'verify', read the current Expo, Android, androidx.webkit or library documentation before coding. Pin dependency versions.
- **Every task ends with** tests, a CHANGELOG line and a check against the phase's acceptance criteria. One feature ID per pull request where practical.
- **No secrets in the repository.** Signing keys and the blocklist signing key live outside the repo.

2. Feature register

Tier shows where the feature came from: **Core** = original spec, **Rec.** = recommended addition, **Opt.** = optional addition. All rows below were selected.

ID	Feature	Tier	Phase
PRV-1	Non-persistent sessions	Core	3
PRV-2	No browsing history	Core	3
PRV-3	Wipe on tab close	Core	3
PRV-4	Clear everything now	Core	3
PRV-5	No analytics or ad SDKs (CI audit)	Core	0
PRV-6	Block third-party cookies	Rec.	3
PRV-7	HTTPS-only mode	Rec.	3
PRV-8	Strip tracking parameters	Rec.	3
PRV-9	Global Privacy Control signal	Rec.	3
PRV-10	Block WebRTC IP leaks	Opt.	3
PRV-11	Reduce fingerprinting	Opt.	3
PRV-12	App lock (expanded, see SEC-2)	Opt.	1
PRV-13	DNS-over-HTTPS (spike first)	Opt.	3
BLK-1	Bundled blocklists	Core	4

ID	Feature	Tier	Phase
BLK-2	Request-level blocking	Core	4
BLK-3	Secure list updates	Core	4
BLK-4	Per-site allowlist	Core	4
BLK-5	Shield toggle and blocked count	Rec.	4
BLK-6	Cosmetic filtering	Rec.	4
BLK-7	Popup and redirect blocking	Rec.	4
BLK-8	'Site broken?' button	Rec.	4
BLK-9	Cookie banner hiding	Opt.	4
BLK-10	Custom rules and list URLs	Opt.	4
VID-1	Detect video elements	Core	5
VID-2	Detect media network requests	Core	5
VID-3	Port existing extension logic	Core	5
VID-4	Download badge	Core	5
VID-5	Video list sheet (thumbnail, title, format, resolution, size)	Core	5
VID-6	Detect videos in iframes	Rec.	5
VID-7	Quality picker	Rec.	5
VID-8	Mark protected streams	Rec.	5
VID-9	Audio-only option	Opt.	5
DL-1	Background downloads	Core	6
DL-2	Pause and resume	Core	6
DL-3	Save to Downloads folder	Core	6
DL-4	Download queue	Rec.	6
DL-5	Merge streaming segments	Rec.	6
DL-6	Auto-resume after interruption	Rec.	6
DL-7	Forward cookies and headers	Rec.	6
DL-8	Check free storage	Rec.	6
DL-9	Clean file names	Rec.	6
DL-10	Downloads screen (library)	Rec.	7
DL-11	Wi-Fi only setting	Opt.	6
WEB-1	Single address bar	Core	2
WEB-2	Navigation controls and progress bar	Core	2
WEB-3	Multiple tabs	Core	2
WEB-4	Search engine choice	Rec.	2
WEB-5	Blank new-tab page	Rec.	2
WEB-6	Find in page	Rec.	2
WEB-7	Share link	Rec.	2
WEB-8	Desktop site toggle	Rec.	2
WEB-9	Permission prompts	Rec.	2
WEB-10	File upload support	Rec.	2
WEB-11	Certificate error page	Rec.	2
WEB-12	New-window handling	Rec.	2
WEB-13	Fullscreen and picture-in-picture video	Opt.	2
WEB-14	Text zoom	Opt.	2
WEB-15	Local bookmarks (stored in encrypted DB)	Opt.	2

ID	Feature	Tier	Phase
APP-1	Expo development build	Core	0
APP-2	react-native-webview engine	Core	0
APP-3	Native blocking module	Core	4
APP-4	Android first, iOS second	Rec.	0
APP-5	Dark mode	Rec.	2
APP-6	Settings screen	Rec.	2
APP-7	Automated tests	Rec.	0-8
APP-8	EAS Build and CI	Rec.	0
APP-9	Local-only crash logs (URLs redacted)	Opt.	8

3. Added security and library requirements

These come from the user's brief and extend the register. They apply to every phase that touches video data.

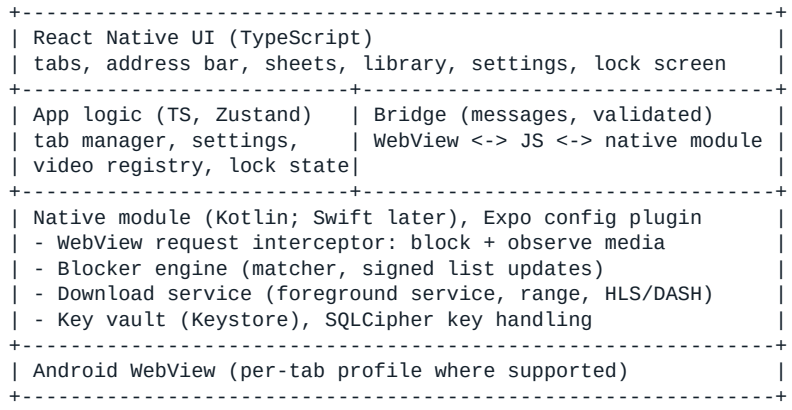
ID	Requirement	Acceptance
SEC-1	The entire SQL database is encrypted (SQLCipher, AES-256). This includes settings, allowlist, bookmarks, video records, download state and thumbnails.	Pull the database file from a test device: it cannot be opened by plain sqlite3 and contains no readable URLs or titles.
SEC-2	The app requires a password or biometric to open. A password is mandatory at first run; biometric is an optional shortcut. Lock on every cold start and after a configurable background timeout.	No screen except the lock screen is reachable while locked. Killing and reopening the app returns to the lock screen.
SEC-3	Video details are never exposed while downloading: generic notification text, random in-progress file names, no titles in logs, blocked screenshots and recents previews.	Notification shows only counts such as '2 downloads'. In-progress files in app storage have UUID names.
SEC-4	Each saved video keeps the original page URL where it was found, and the user can open that page from the library.	Tapping 'Open source page' opens the stored page in a new tab.
SEC-5	After downloading, the user can remove the data about a video (title, URLs, thumbnail, headers), with or without deleting the file. Optional setting: remove details automatically when a download completes.	After removal, a decrypted dump of the database holds no trace of that video.
EXT-1	The user supplies the source code of the existing video detection and download extension. It is audited and ported in Phase 5.	Code received and an audit note written before Phase 5 starts.

Derived items to confirm with the owner

- **D-1** Block screenshots and recents previews (Android FLAG_SECURE), on by default, switchable in settings. 'Hide previews in app switcher' was not selected, but SEC-3 requires it in practice.
- **D-2** Notification visibility set to private; progress text never names the video.
- **D-3** Files in the public Downloads folder are not encrypted and carry the clean file name (DL-3, DL-9). This follows the original spec. See Risks.

4. Architecture and tech stack

Layers



Tech stack

Area	Choice	Notes
Framework	Expo (current stable SDK), development builds, config plugins, TypeScript strict	Expo Go cannot load the custom native module (APP-1). Verify SDK and minSdk before starting.
Web engine	react-native-webview, wrapped or patched	Needs request-level interception on Android. Options: patch with patch-package, or own native view. Decide in Phase 0.
State / routing	Zustand, Expo Router	Keep state small. Never persist anything outside the encrypted DB.
Database	expo-sqlite with SQLCipher enabled (verify current config option), fallback op-sqlite with SQLCipher	Raw 256-bit key. PRAGMA secure_delete=ON.
Keys and unlock	expo-secure-store / Android Keystore, expo-local-authentication, Argon2id via a vetted library (verify)	See key hierarchy in Phase 1.
Blocker engine	Phase 4a: own Kotlin matcher over EasyList/EasyPrivacy network rules. 4b if needed: adblock-rust (Brave, MPL-2.0) via JNI	Start simple, measure, then decide.
Downloads	Custom Kotlin foreground service (OkHttp, range requests); iOS URLSession later	Expo file APIs are not enough for reliable background, pausable downloads.
Muxing	Android MediaExtractor/MediaMuxer for segments to MP4	ffmpeg-kit was retired upstream: use it only through a maintained fork you have verified.
Testing	Jest, React Native Testing Library, JUnit (Kotlin), Maestro for E2E	Run unit tests on every commit.
CI/CD	GitHub Actions, EAS Build	Includes the tracker-dependency audit (PRV-5).

Folder structure

```
app/                        Expo Router screens (browser, library, settings, lock)
src/browser/                tabs, address bar logic, search, bookmarks
src/privacy/                session wipe, https-only, param stripping, GPC
src/blocker/                list loader, update client, allowlist, cosmetic scripts
src/video/                  detector bridge, registry, probe, quality picker
src/downloads/              queue client, state mapping, library actions
src/security/               lock, key wrap/unwrap client, auto-lock timer
src/db/                     schema, migrations, repositories
src/injected/               scripts injected into pages (bundled with esbuild)
modules/private-browser-native/   Kotlin module + Expo config plugin
scripts/                    dependency audit, list compiler, signing tools
docs/                       this plan, CHANGELOG, extension audit notes
```

5. Delivery phases

Size guide: S = under 2 days, M = about a week, L = 2 weeks or more, for one developer with an AI agent. Each phase ends with its acceptance criteria met.

Phase 0: Foundations (size S)

Goal. A running development build, CI and project rules.

Tasks

- Create the Expo project (TypeScript strict, lint, formatter). Build an Android development build (APP-1). Choose the WebView approach: patch react-native-webview or own native view (APP-2).
- Create the native module skeleton and config plugin. Set Android flags: allowBackup=false, WebView file access off, no mixed content.
- Set up GitHub Actions: lint, tests, EAS build. Add the dependency audit script with a deny-list of analytics and ad SDKs that fails the build (PRV-5).
- Decide and record: Android first, iOS later (APP-4). Create the folder structure and CHANGELOG.

Acceptance criteria

✓ Dev build installs on a physical Android device and loads a URL in a WebView.
✓ CI runs green; adding a deny-listed package makes CI fail.
✓ Decision notes for the WebView approach committed in docs/.

Phase 1: Security foundation (size M)

Goal. Encrypted database and mandatory lock before any user data exists (SEC-1, SEC-2, PRV-12).

Tasks

- Key hierarchy: generate a random 32-byte database key (DBK) on first run. Password path: derive a key-encryption key with Argon2id (tune to about 300 to 500 ms on a mid-range phone), wrap the DBK with AES-256-GCM, store wrapped blob, salt and parameters. Biometric path: a second wrapped copy using an Android Keystore key that requires user authentication and is invalidated by new biometric enrollment.
- Keep the DBK in native memory only. On lock: close the database and drop the key. The JS layer never sees the raw key.
- First-run flow: create password (minimum length enforced), optional biometric. Lock screen on cold start and after a background timeout (setting).
- Wrong-attempt backoff (increasing delay). No password recovery: 'forgot password' offers a full data wipe. Explain this at setup.
- Schema v1 with migrations (section 6). Enable secure_delete. Block screenshots with FLAG_SECURE (D-1).

Acceptance criteria

✓ Database pulled from the device cannot be read without the key (verify with sqlite3 and strings).
✓ Nothing but the lock screen is reachable when locked, including via deep links and notifications.
✓ Biometric unlock works and stops working after the biometric set changes, falling back to password.
✓ Five wrong passwords trigger increasing delays.

Phase 2: Browser shell (size M)

Goal. A usable browser with navigation, tabs and settings (WEB-1 to WEB-15, APP-5, APP-6).

Tasks

- Address bar: URL versus search detection, search engine choice with DuckDuckGo default (WEB-1, WEB-4). Navigation controls, progress bar, pull to refresh (WEB-2).
- Tab manager with switcher, open, close, and a blank new-tab page (WEB-3, WEB-5). Handle target=_blank and window.open as new tabs (WEB-12).
- Find in page, share link, desktop site toggle, text zoom (WEB-6, WEB-7, WEB-8, WEB-14).

- Permission prompts for camera, microphone, location with per-request consent (WEB-9). File upload and camera input (WEB-10). Certificate error interstitial (WEB-11).
- Fullscreen video and picture-in-picture (WEB-13). Bookmarks stored in the encrypted DB (WEB-15). Dark mode and settings screen skeleton (APP-5, APP-6).

Acceptance criteria

✓ All WEB features work on three test sites including a login form, a video page and a file upload page.

✓ Closing a tab frees its WebView. Bookmarks survive restart and are absent from a raw database dump.

Phase 3: Privacy layer (size M)

Goal. Nothing persists; tracking surface reduced (PRV-1 to PRV-13).

Tasks

- Per-tab isolation. Android: use androidx.webkit profiles (one per tab, deleted on close) where the installed WebView supports it; fallback is a shared store wiped when the last tab closes plus per-origin cookie removal. iOS later: WKWebsiteDataStore.nonPersistent() (PRV-1, PRV-3).
- Never write history (PRV-2). 'Clear everything now' removes cookies, cache, WebStorage, IndexedDB, permissions, form data and open tabs (PRV-4).
- Block third-party cookies (PRV-6). HTTPS-only: upgrade http, show an interstitial with 'continue over HTTP' if the secure load fails (PRV-7).
- Strip tracking parameters (utm_*, fbclid, gclid, and a maintained list) before navigation (PRV-8). Send Sec-GPC: 1 on main-frame requests and set navigator.globalPrivacyControl (PRV-9).
- WebRTC: injected script forces relay-only ICE or disables RTCPeerConnection, with a settings toggle because it breaks calls (PRV-10). Fingerprinting: modest, low-breakage reductions for canvas, audio and hardware values, with a toggle (PRV-11).
- DoH spike (PRV-13): WebView has no direct DoH setting. Evaluate (a) detecting and guiding the user to Android Private DNS, and (b) a local proxy using ProxyController with DoH resolution. Ship (a) unless (b) proves stable.

Acceptance criteria

✓ After closing all tabs, a test site's cookies and storage are gone.

✓ A test page reports no third-party cookies and the stripped URL has no tracking parameters.

✓ WebRTC leak test shows no local or public IP exposure when enabled.

✓ DoH decision documented.

Phase 4: Ad and tracker blocker (size L)

Goal. Blocking inside the WebView with updates and allowlist (BLK-1 to BLK-10, APP-3).

Tasks

- List compiler (scripts/): fetch EasyList, EasyPrivacy and the cookie-notice list at build time, parse network rules, output a compact binary format bundled in the app (BLK-1). Keep license notices (EasyList is CC BY-SA / GPL) in an About screen.
- Matcher in Kotlin: host suffix trie, URL substring and simple pattern rules, third-party check using the public suffix list. Blocked requests return an empty response from the interceptor (BLK-2). Measure speed and memory before considering adblock-rust.
- Signed updates: lists hosted over HTTPS, Ed25519 signature checked against a public key embedded in the app, atomic swap, rollback on failure, background refresh (BLK-3).
- Allowlist by host stored in the encrypted DB (BLK-4). Toolbar shield with per-page blocked count held in memory only (BLK-5). 'Site broken?' reloads the tab once with blocking off (BLK-8).
- Cosmetic filtering: generic and site-specific CSS injected before content loads, with a selector budget for performance (BLK-6). Cookie banner hiding from the cookie-notice list (BLK-9).

- Popup and redirect blocking: only allow new windows and redirects tied to a user gesture (BLK-7). Custom rules and extra list URLs, validated and size-capped; user lists are unsigned and labelled as such (BLK-10).

Acceptance criteria

✓ A set of 20 known ad-heavy pages loads with ad domains blocked and the page still usable.
✓ Matcher p95 under 1 ms per request after warm-up on a mid-range device; list memory under about 30 MB.
✓ A tampered list file is rejected and the previous list stays active.
✓ Allowlisting a site restores its blocked content.

Phase 5: Video detection (blocked until EXT-1) (size L)

Goal. Reliable detection using the user's existing extension logic (VID-1 to VID-9).

Tasks

- EXT-1 intake: read every file of the extension, write docs/extension-audit.md listing behaviors, permissions, chrome.* APIs used, endpoints contacted and edge cases it handles. Remove or flag anything that phones home.
- Port using the mapping in section 7. Bundle injected scripts with esbuild. Inject into all frames (VID-1, VID-6, VID-3).
- Network observation in the native interceptor: record .mp4, .m3u8, .mpd and related media requests with headers, without blocking them (VID-2).
- Metadata probe: HEAD or ranged GET for type and size; parse HLS master playlists and DASH manifests for variants and resolutions; title from og:title, document.title or the element; thumbnail from poster or a captured frame (VID-5, VID-7).
- Protected content: detect EME use, SAMPLE-AES keys and ContentProtection tags and show 'DRM protected' (VID-8). blob: sources cannot be fetched directly; rely on the underlying manifest or segment requests.
- UI: toolbar badge with count (VID-4) and a bottom sheet listing thumbnail, title, format, resolution and size (VID-5). Audio-only choice where an audio rendition exists (VID-9).
- Each detected item stores the top-level page URL and the frame URL at detection time (needed for SEC-4).

Acceptance criteria

✓ Test pages pass: direct MP4, HLS with variants, DASH, video inside an iframe, DRM sample (flagged), blob/MSE page (documented limitation).
✓ Detection results match the original extension on a shared list of at least 10 real pages the owner chooses.
✓ Every item carries its source page URL.

Phase 6: Download engine (size L)

Goal. Reliable, pausable, private downloads (DL-1 to DL-9, DL-11, SEC-3).

Tasks

- State machine: queued, probing, downloading, paused, merging, completed, failed, canceled. Persist state and per-segment progress in the encrypted DB while unlocked.
- Kotlin foreground service (declare the correct foreground service type for the Android version; request POST_NOTIFICATIONS on Android 13+; verify current rules). Range requests with ETag/If-Range, retries with backoff, resume after network loss or restart (DL-1, DL-2, DL-6).
- Queue with a limit of 2 concurrent downloads and 4 parallel segments (DL-4). Wi-Fi only option (DL-11). Free-space check before start (DL-8).
- Forward Cookie, Referer and User-Agent captured at detection (DL-7). Keep them in memory and in encrypted DB only until completion, then purge.
- HLS/DASH: download segments, then merge to MP4 with MediaExtractor/MediaMuxer (DL-5).

- Privacy: in-progress files live in app-private storage with UUID names; notification text is generic (SEC-3). While locked, active downloads continue from in-memory state; if the process dies the queue resumes after the next unlock.
- On completion: write to the Downloads collection via MediaStore with a sanitised, length-limited, de-duplicated name from the title (DL-3, DL-9), then delete temp files.

Acceptance criteria

✓ Pause, kill the app, unlock and resume completes a 1 GB file with a correct checksum.
✓ Toggling airplane mode mid-download recovers.
✓ Low-storage case shows a clear error before starting.
✓ Notifications and temp file names never contain titles. Completed file appears in the system Downloads with a clean name.

Phase 7: Video library (size M)

Goal. A private list of downloads with source links and removal controls (DL-10, SEC-4, SEC-5).

Tasks

- Library screen (reads only from the encrypted DB): thumbnail, title, size, resolution, date, status and the source site's host.
- Actions per item: play (system player through a content URI), open source page in a new tab (SEC-4), share file, retry failed, remove details only, delete file and details (SEC-5).
- Bulk select and 'Clear all video data'. Setting: remove details automatically when a download completes (default off).
- Removal deletes rows, thumbnail data and stored headers, then runs a WAL checkpoint and periodic VACUUM. Files that were kept stay in Downloads without any record in the app.

Acceptance criteria

✓ Open source page loads the exact stored page URL in a new tab.
✓ After 'remove details', a decrypted dump of the database contains no URL, title or thumbnail for that video.
✓ Auto-remove setting works and does not affect the saved file.

Phase 8: Hardening and release (size M)

Goal. Tests, security review and a signed build (APP-7, APP-8, APP-9).

Tasks

- Complete unit, integration and E2E coverage from section 9. Run the security review against section 8.
- Local-only crash logs: redact URLs and titles, store encrypted, allow manual export by the user (APP-9).
- Performance pass: cold start, tab memory, blocker latency, battery during a long download.
- Signed release build through EAS. Decide distribution channel (see Risks).

Acceptance criteria

✓ All acceptance criteria from phases 0 to 7 pass on at least three physical Android devices.
✓ Threat model checklist has no open high items.

6. Data model (encrypted SQLite)

```
settings(key TEXT PRIMARY KEY, value TEXT)
allowlist(host TEXT PRIMARY KEY, added_at INTEGER)
bookmarks(id INTEGER PRIMARY KEY, url TEXT, title TEXT, added_at INTEGER)
custom_lists(id INTEGER PRIMARY KEY, kind TEXT, source TEXT, content BLOB, updated_at INTEGER)

videos(
  id TEXT PRIMARY KEY,            -- uuid
```

```
  source_page_url TEXT NOT NULL,  -- top-level page where found (SEC-4)
  frame_url TEXT,
  media_url TEXT NOT NULL,
  title TEXT, format TEXT, resolution TEXT, size_bytes INTEGER,
  protected INTEGER DEFAULT 0,
  thumbnail BLOB,
  detected_at INTEGER
)
downloads(
  id TEXT PRIMARY KEY,
  video_id TEXT REFERENCES videos(id) ON DELETE CASCADE,
  state TEXT, bytes_done INTEGER, segments_done INTEGER, segments_total INTEGER,
  temp_name TEXT,                 -- uuid, no title
  headers_json TEXT,              -- purged on completion
  file_uri TEXT, final_name TEXT,
  error TEXT, started_at INTEGER, completed_at INTEGER
)
PRAGMA secure_delete = ON;  PRAGMA foreign_keys = ON;
```

Removal rules: 'remove details only' deletes the videos row (cascading to downloads) and keeps the file. 'Delete file and details' also deletes the file through its URI. Neither writes anything to logs.

7. Porting the existing video extension

Chrome and Brave extensions cannot run inside a mobile WebView, so the working logic is moved, not loaded. Send the full source (manifest.json, content scripts, background or service worker, popup, any helper libraries). The agent then follows this mapping.

Extension part	Becomes in the app
content_scripts (page DOM, video tags)	Injected scripts in src/injected/, run in all frames before and after load
chrome.runtime.sendMessage / onMessage	window.ReactNativeWebView.postMessage with a validated message schema
background service worker logic	TypeScript in src/video/ (registry, probe, decisions)
chrome.webRequest (media URL sniffing)	Native request observer in the Kotlin interceptor; on iOS, JS hooks for fetch/XHR and PerformanceObserver (weaker)
chrome.downloads	The download engine in modules/private-browser-native
chrome.storage	Encrypted SQLite repositories
chrome.tabs / tab info	Tab manager (page URL and title come from the tab state)
Popup UI	The video bottom sheet (VID-5) and download badge (VID-4)
Remote endpoints, analytics	Remove, or raise to the owner for a decision

Deliverable of the audit: docs/extension-audit.md with a table of every behavior and where it lands in the app. The detection results of the port are compared with the original extension on the owner's sample pages.

8. Threat model

Threat	Mitigation	Phase
Phone lost or borrowed, app closed	Mandatory unlock; SQLCipher database; key only in Keystore-wrapped form	1
Someone glances at recents, screen recording or screenshots	FLAG_SECURE, generic notifications, lock on background timeout	1, 6
Other apps read the Downloads folder	Accepted per spec; named risk; optional app-private storage mode is an open decision	10
Notification reveals video titles	Generic text only; private lock-screen visibility	6
Database leftovers after removal	secure_delete, checkpoint, VACUUM; test with a decrypted dump	7
Android backup extracts data	allowBackup=false, no cloud backup rules	0
Malicious page abuses the JS bridge	Bridge only for known message types, payload validation, origin checks, no addJavascriptInterface exposure of sensitive methods	2, 5

Threat	Mitigation	Phase
Poisoned blocklist update	Ed25519 signature check, atomic swap, rollback	4
Local file or mixed-content attacks	WebView file access off, mixed content blocked	0
Logs leak browsing data	Never log URLs or titles; crash logs redacted and encrypted	8
Dependency adds trackers	CI deny-list audit, pinned versions	0

9. Testing and QA

- **Unit (Jest, JUnit):** URL vs search detection, tracking-parameter stripping, matcher rules, list signature check, key wrap/unwrap, filename sanitising, download state machine, removal queries.
- **Integration:** bridge messages, database migrations, detector against saved HTML fixtures, manifest parsing for HLS and DASH.
- **E2E (Maestro):** first-run setup, lock and unlock, open site, block count, detect video, download, pause, resume, open source page, remove details.
- **Test pages:** direct MP4, HLS with variants, DASH, iframe embed, blob/MSE, DRM sample, ad-heavy page, cookie banner page, login form, upload form.
- **Device matrix:** at least three physical Android devices across low, mid and high memory, and two Android versions including the newest.
- **Security tests:** database cannot be read without key; no titles in notifications, logs or temp file names; decrypted dump after removal is clean.

10. Risks and open decisions

#	Item	Detail / proposed handling
1	Extension source pending	Phase 5 is blocked until supplied (EXT-1). Phases 0 to 4 and 7 UI work can proceed.
2	Files in public Downloads are not encrypted	Matches the original spec. Option to add a 'save inside the app, encrypted' mode needs the owner's decision and is not in v1.
3	Forgot password means data wipe	No recovery path by design, since recovery weakens encryption. Tell the user at setup.
4	Distribution	Google Play restricts apps that download video from services without permission. Plan on APK or F-Droid-style distribution and check current policy before choosing.
5	DRM and site terms	DRM streams are never downloaded. Users are responsible for respecting copyright and site terms.
6	blob/MSE videos	Cannot be downloaded from the blob URL. Works only when the underlying manifest or segments are observed.
7	iOS is different	No request-level blocking in WKWebView: use WKContentRuleList (rule-count limits); weaker network observation; background download limits; no foreground service. Plan iOS as a separate phase after Android.
8	Per-tab profiles need a recent WebView	androidx.webkit profile support depends on the installed WebView. Fallback wipe behavior is defined in Phase 3.
9	ffmpeg-kit retired	Use platform muxers; adopt a fork only after verifying it is maintained.
10	DNS-over-HTTPS may not be feasible in WebView	Spike in Phase 3 with a defined fallback.

Appendix: agent kickoff prompt

Paste this at the start of every agent session, then name the phase and feature IDs to work on.

```
You are building the Private Browser (Expo / React Native, Android first).
Read docs/implementation-plan.pdf fully before writing code.
Rules: implement only features listed in the feature register; follow the phase
order; respect the security rules in section 1 (encrypted DB, no analytics, no
logging of URLs or titles, generic notifications). Where the plan says 'verify',
check current official docs first and pin versions. Reference feature IDs in
```

```
commits. Finish each task with tests, a CHANGELOG line and a check against the
phase acceptance criteria. If something is unclear or missing, stop and ask
instead of inventing behavior.
Current task: Phase __ , features __ .
```